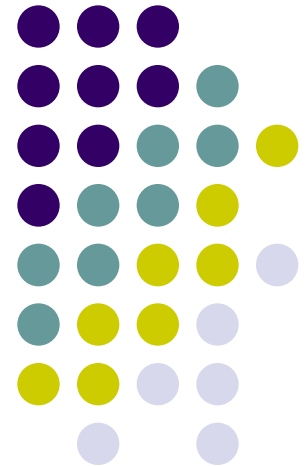


# Making a Machine Learning Model with Pipeline & GridSearchCV

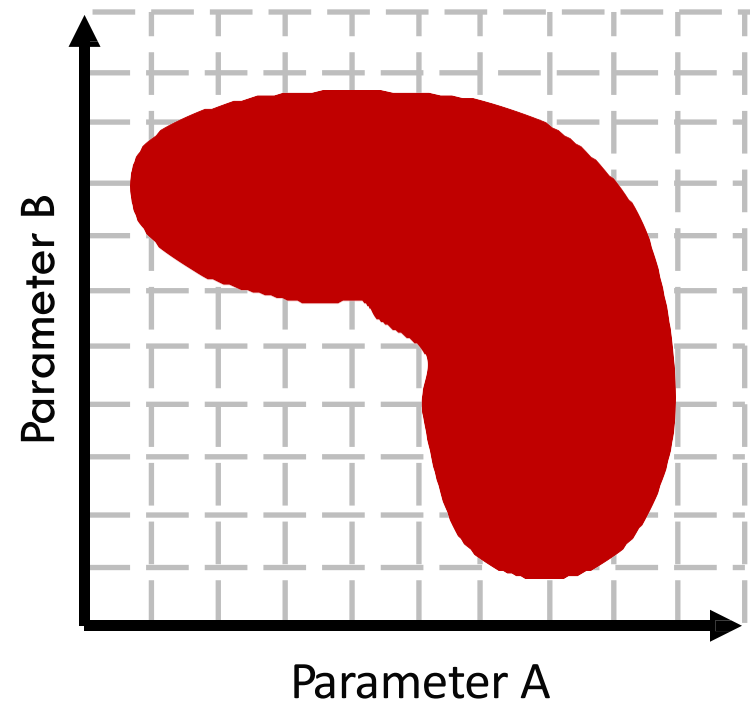
Machine Learning in Practice I  
Corvinus University of Budapest, 2025



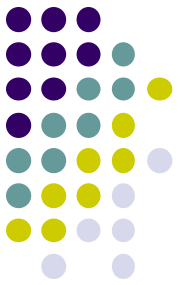
# Generalized Hyperparameter Grid Search



- Hyperparameter selection for regularization / better models requires cross validation on training data
- Linear and logistic regression methods have classes devoted to grid search
- Grid search can be useful for other methods too, so a generalized method is desirable
- Scikit-learn contains `GridSearchCV`, which performs a grid search with parameters using cross validation



# Grid Search with Cross Validation: The Syntax



**Import the class containing the grid search method**

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
```

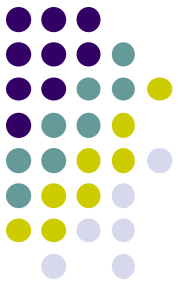
**Create an instance of the estimator and grid search class**

```
LR = LogisticRegression(penalty='l2')
GS = GridSearchCV(LR, param_grid={'c':[0.001, 0.01, 0.1]},
                  scoring='accuracy', cv=4)
```

**Fit the instance on the data to find the best model, and then predict**

```
GS = GS.fit(X_train, y_train)
y_train = GS.predict(X_test)
```

# Optimizing the Rest of the Pipeline

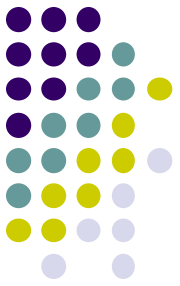


- Grid searches enable model parameters to be optimized
- How can this be incorporated with other steps of the process (e.g. feature extraction and transformation)?

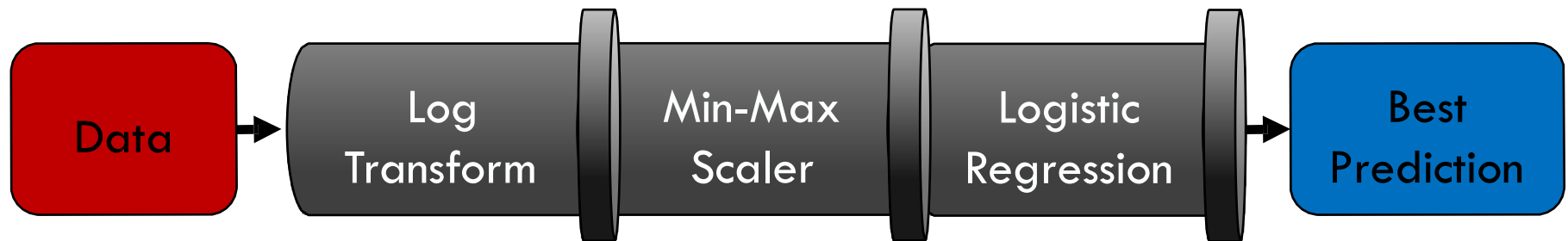


**Pipelines!**

# Automating Machine Learning with Pipelines

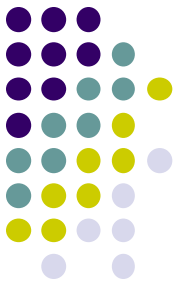


- Machine learning models often selected empirically
- By trying different processing methods and tuning multiple models

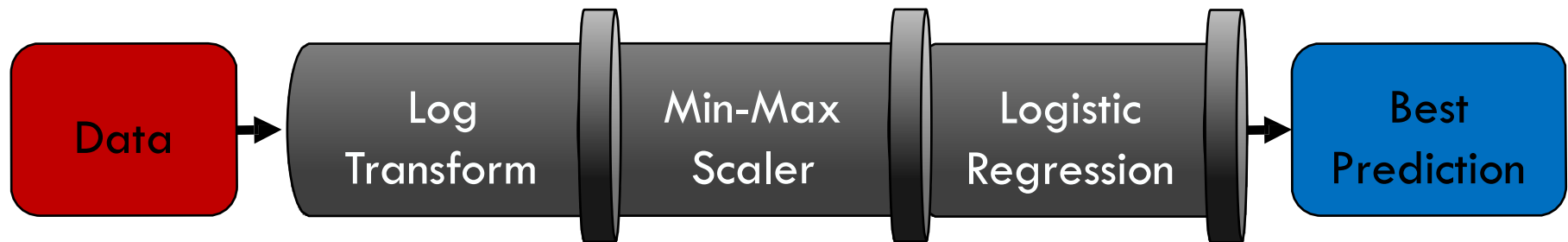


**How to automate this process?**

# Automating Machine Learning with Pipelines

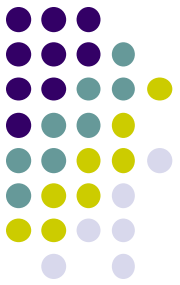


- Pipelines in Scikit-Learn allow feature transformation steps and models to be chained together
- Successive steps perform 'fit' and 'transform' before sending data to the next step



**Pipelines make automation and reproducibility easier!**

# Pipelines: The Syntax



**Import the class containing the pipeline method**

```
from sklearn.pipeline import Pipeline
```

**Create an instance of the class with estimators**

```
estimators = [('scaler', MinMaxScaler()), ('lasso', Lasso())]
```

```
Pipe = Pipeline(estimators)
```

**Fit the instance on the data and then predict the expected value**

```
Pipe = Pipe.fit(X_train, y_train) y_predict =
```

```
Pipe.predict(X_test)
```

**Features can be combined from different transform methods using `FeatureUnion`**

